



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

软件工程课程作业

软件设计报告

丛方昊 2310682

杨凝霜 2314009

罗雅淇 2313483

钱展飞 2312479

王恩业 2312404

2026 年 6 月 8 日

目录

一、 系统总体设计	1
(一) 项目背景与目标	1
(二) 系统功能概述	1
(三) 系统总体架构设计	1
1. 架构风格说明	1
2. 系统整体结构图	2
(四) 技术选型说明	2
1. 开发语言、框架、数据库与中间件	2
2. 选型理由分析	3
二、 详细设计	4
(一) 模块划分与模块功能设计	4
1. 模块职责说明	4
2. 模块之间的调用关系	6
(二) 核心业务流程设计	7
(三) 接口设计	8
三、 UML 设计	11
(一) 用例图 (Use Case Diagram)	11
(二) 类图 (Class Diagram)	12
1. 领域模型类图	12
2. 服务层类图	13
(三) 顺序图 (Sequence Diagram)	14
1. 文档实时协作编辑 (UC-05)	14
2. 版本历史与回滚 (UC-07)	15
3. 白板实时协作 (BoardPage)	16
(四) 状态图与部署图	17
1. 文档协作会话状态图	17
2. 部署图	18
四、 数据库设计	19
(一) 数据库 E-R 图	19
(二) 主要数据表设计	20
1. 用户表 User	20
2. 协作项目表 CollaborativeItem	20
3. 权限表 Permission	21
4. 版本快照表 Version	22
5. 评论表 Comment	22
6. 枚举类型说明	24
五、 其他内容	24
(一) 安全性设计	24
(二) 性能优化设计	24
(三) 可扩展性设计	24

(四) 异常处理机制 25

(五) 日志与监控方案 25

一、 系统总体设计

(一) 项目背景与目标

CollabPlatform 是一个面向课程实践的多人实时协作平台，目标是为小组协作提供两类核心工作空间：**共享白板**与**共享文档**。在典型的团队场景中，白板用于头脑风暴、流程梳理与原型草图；文档用于需求/方案/会议纪要的沉淀与多人联动编辑。系统采用前后端分离的 Web 架构，通过 HTTP 提供数据管理接口，通过 WebSocket 提供实时协作通道。

本项目的总体目标如下：

- (1) **协作一致性**：多人同时编辑时尽量避免冲突覆盖，保证内容可合并、可回滚。
- (2) **可用性与易上手**：浏览器即可使用，界面统一（基于 antd），交互直观（工具栏、版本侧边栏、在线状态提示）。
- (3) **工程化与可维护**：后端采用“routes → controllers → services”分层；前端采用模块化目录与统一封装（request/useSocket/authStore），便于多人并行开发与后续迭代。
- (4) **安全与权限**：统一鉴权（JWT），并在业务层与 Socket 层对访问权限进行校验（Owner/Editor/Viewer）。

(二) 系统功能概述

系统从用户视角提供以下主要功能：

- (1) **用户认证**：注册、登录、刷新 token、获取当前用户信息。
- (2) **协作项目**：以 CollaborativeItem 为统一载体，支持 Whiteboard 与 Document 两种类型（同一套权限模型与版本/评论扩展能力）。
- (3) **共享白板**：基于 Fabric.js 的画布编辑（选择、手绘、矩形/圆形/箭头/文字、拖拽缩放、删除、导出 PNG），支持多人协作与多人光标显示；并支持保存当前状态与版本快照、版本回滚。
- (4) **共享文档**：基于 TipTap 的富文本编辑（标题、列表、代码块、表格、图片、链接等），基于 Yjs 的 CRDT 协作编辑与协作光标；支持评论/回复、版本快照与回滚、导出 Markdown/HTML 等扩展能力。
- (5) **实时协作基础设施**：统一的 Socket 连接管理、房间管理（item:<itemId>）、广播与断线重连状态提示。

系统采用统一响应格式（{code,data,message}）与错误码约定（40001/40101/40301/40401/40901/50000），前端通过 axios 拦截器统一解包与错误提示，减少业务层重复代码。

(三) 系统总体架构设计

1. 架构风格说明

系统整体采用 B/S（Browser/Server）架构与前后端分离模式，前端为单页应用（SPA），后端提供 RESTful 风格的 HTTP API 与 Socket.io 实时通道。架构中同时存在两条数据通路：

- (1) **HTTP（请求-响应）**：用于登录注册、项目数据增删改查、版本/评论等需要落库的操作，强调幂等与可追踪。

- (2) **WebSocket (长连接)**: 用于实时协作同步 (文档/白板的增量更新、协作光标/在线状态等), 强调低延迟与广播效率。

后端内部采用分层架构以提升可维护性与可测试性:

routes (声明路径) → controllers (处理 req/res) → services (业务逻辑) → Prisma (数据访问) → PostgreSQL

在实时协作层面, 文档协作使用 **Yjs (CRDT)** 实现无冲突合并; 白板协作采用 “Fabric 事件 ↔ Yjs 数据结构” 的双向绑定, 将画布元素抽象为可合并的数据更新, 从而在多人并发操作时避免简单 “整页快照覆盖” 带来的冲突。

2. 系统整体结构图

Browser (React + Vite)

```

|
|-- HTTP (axios) -----> Express /api/*
|
|                               |
|                               |-- routes -> controllers -> services
|                               |-- Prisma -> PostgreSQL
|
|-- WebSocket (socket.io-client) ---> Socket.io Server
|
|                               |-- 连接鉴权 (JWT)
|                               |-- 房间: item:<itemId>
|                               |-- 广播: broadcastToRoom (排除发送者)
|                               |-- 协作事件: doc:* / board:* / cursor

```

其中, 前端通过 `request.js` 统一封装 HTTP 请求 (自动附带 `accessToken`、统一错误处理); 通过 `useSocket` 统一封装 WebSocket (单例连接、自动重连、连接状态枚举、`join/leave/ping` 等通用方法)。

(四) 技术选型说明

1. 开发语言、框架、数据库与中间件

前端:

- React 18 + Vite: 构建单页应用与工程化开发。
- react-router-dom v6: 路由与受保护页面跳转 (ProtectedRoute)。
- antd: 统一 UI 组件库与交互规范。
- Zustand: 轻量状态管理 (登录态持久化到 `localStorage`)。
- axios: HTTP 客户端与拦截器封装。
- socket.io-client: 实时连接与自动重连。
- TipTap (ProseMirror): 文档富文本编辑器。

- Yjs + y-protocols + y-prosemirror: CRDT 协作内核与 Awareness（协作光标/在线用户）。
- Fabric.js: 白板画布与图形对象模型。

后端:

- Node.js + Express: HTTP API 服务与中间件体系。
- Prisma: ORM 与数据库模型管理。
- PostgreSQL: 关系型存储（可部署于 Supabase）。
- Socket.io: 实时通信与房间广播。
- jsonwebtoken + bcrypt: 鉴权与密码哈希。
- Redis（可选）: 缓存/限流/计数等能力，未配置时自动降级。
- morgan: 请求日志。

2. 选型理由分析

- (1) 协作一致性: Yjs 作为成熟的 CRDT 方案，能够在弱网络与并发输入下实现无冲突合并；Awareness 机制可自然承载光标/在线信息同步。
- (2) 开发效率: React + Vite 具备快速热更新与良好生态；antd 提供覆盖全面的组件，减少重复造轮子。
- (3) 可维护性: Express 的中间件模式与“routes/controllers/services”分层清晰；Prisma 通过 schema 管理数据模型与迁移，降低手写 SQL 的出错概率。
- (4) 实时与可靠: Socket.io 兼容性好且内置重连机制，适合课程项目的多环境部署；房间广播可将协作范围限制在单项目内，降低不必要的网络开销。
- (5) 可扩展: CollaborativeItem 抽象统一了“白板/文档”两类资源，后续新增协作类型（如表格、脑图）只需扩展 type 与对应模块即可复用权限、版本、评论等基础能力。

二、 详细设计

(一) 模块划分与模块功能设计

1. 模块职责说明

系统按照前后端分离的结构，将功能划分为若干具有清晰职责边界的模块，模块之间通过明确定义的接口交互。

后端模块划分 (**server/src/**):

(1) 配置层 (**config/**): 负责运行环境初始化，是整个服务启动的前置依赖。

- **env.js**: 加载 `.env`，校验 `DATABASE_URL`、`JWT_SECRET` 等关键变量；缺失时直接退出进程，防止以错误配置启动。
- **prisma.js**: 导出 `PrismaClient` 单例，开发模式下挂到 `global`，避免热重载时重复建立数据库连接。
- **redis.js**: 连接 Redis；若 `REDIS_URL` 为空或连接失败，导出空操作 `Proxy` 进行容错降级，业务层无需感知 Redis 是否可用。

(2) 工具层 (**utils/**): 提供全局通用能力，供各层调用。

- **AppError.js**: 自定义错误类，携带 `httpStatus`、业务 `code`、`message` 三个字段；业务层 `throw` 后由错误中间件统一捕获并格式化返回。
- **response.js**: 封装 `success(res, data)` 与 `fail(res, status, code, msg)` 两个函数，所有接口必须通过它们返回，保证响应格式一致。
- **jwt.js**: 封装 `signAccessToken`、`signRefreshToken`、`verifyToken`；有效期通过环境变量 `JWT_EXPIRES_IN`/`JWT_REFRESH_EXPIRES_IN` 配置；校验失败直接抛 `AppError(401)`。

(3) 中间件层 (**middlewares/**): 处理横切关注点，以管道形式插入 Express 请求链路。

- **requestLogger.js**: 基于 `morgan`，记录每个请求的方法、路径、状态码与耗时。
- **auth.js**: JWT 鉴权中间件，从 `Authorization: Bearer <token>` 头提取并校验 `token`，成功后将 `{ userId }` 挂载到 `req.user`，供控制器读取。
- **errorHandler.js**: 全局错误处理，放在所有路由之后；捕获 `AppError` 按其字段格式化返回，未知错误统一返回 `50000`，开发环境输出堆栈信息。

(4) 路由层 (**routes/**): 只负责声明 URL 路径与 HTTP 方法，将请求转发给对应 `controller`，本身不含任何业务逻辑。包含 `auth.routes.js` (认证)、`boards.routes.js` (白板)、`doc.routes.js` (文档)、`items.routes.js` (项目管理)、`health.routes.js` (健康检查)，统一在 `routes/index.js` 挂载到 `/api` 前缀。

(5) 控制器层 (**controllers/**): 负责处理 HTTP 请求与响应，从 `req` 中提取参数、调用 `service`，再通过 `response` 工具返回结果；不包含业务逻辑判断。包含 `auth.controller.js`、`board.controller.js`、`doc.controller.js`、`item.controller.js`。

(6) 业务层 (**services/**): 核心业务逻辑所在层，只负责业务计算、数据库操作与权限校验，不感知 HTTP，可被多个 `controller` 复用，也方便编写单元测试。

- `auth.service.js`: 注册 (字段校验、唯一性检查、`bcrypt` 哈希)、登录 (密码比对、状态检查、`token` 签发)、`token` 刷新。
- `board.service.js`: 白板 CRUD, 提供 `assertBoardReadable/assertBoardWritable/assertBoardOwner` 三个权限断言函数, 以及版本快照管理 (保留最近 50 条)。
- `doc.service.js`: 文档 CRUD, 提供 `assertDocumentAccess(itemId, userId, minRole)` 权限断言, 以及评论 (含回复树与锚点定位)、版本快照与回滚、成员邀请与权限调整的完整业务逻辑。
- `item.service.js`: 协作项目的通用管理操作, 支持按类型筛选列表与权限更新。

(7) **Socket 层 (`socket/`)**: 实时协作通信层, 与 HTTP 层并行工作。

- `index.js`: 初始化 `Socket.io` 并与 HTTP Server 绑定; 在 `io.use()` 中间件中对握手 `token` 进行 JWT 校验, 校验通过后将 `userId` 挂载到 `socket.data`; 导出 `joinRoom`、`leaveRoom`、`broadcastToRoom` (排除发送者) 三个通用方法。
- `handlers.js`: 为每条连接注册业务事件, 包括房间加入/离开 (查询 `item` 类型后分发到白板或文档权限校验)、文档协作事件 (`doc:operation` / `doc:cursor` / `doc:title-changed` / `doc:sidebar-changed`)、白板协作事件 (`board:sync` / `board:draw` / `board:cursor`)。每个写操作事件均在广播前调用对应 `service` 的权限断言函数。

前端模块划分 (`client/src/`):

- (1) **HTTP 封装 (`api/`)**: 基于 `axios` 创建实例, 请求拦截器自动注入 `Authorization` 头; 响应拦截器统一解包 (`code === 0` 直接返回 `data`)、错误提示 (`Toast`) 与自动登出 (`code === 40101`)。按业务领域进一步封装: `auth.api.js` (注册/登录/刷新)、`board.api.js` (白板 CRUD)、`doc.api.js` (文档 CRUD/评论/版本/成员)。
- (2) **WebSocket 封装 (`socket/useSocket.js`)**: 单例模式封装 `socket.io-client`, 保证多组件共享同一连接; 暴露 `connect`、`disconnect`、`joinRoom`、`leaveRoom`、`emit`、`on`、`off`、`ping` 等方法; 维护 `status` 枚举 (`disconnected` / `connecting` / `connected` / `reconnecting`), 供 `App.jsx` 渲染全局连接状态横幅。
- (3) **协作层 (`collab/`)**: 文档 CRDT 协作内核, 不依赖第三方协作服务器。`socketIoYjsProvider.js` 将 `Yjs` 增量 `update` 与 `Awareness` 编码为 `base64` 后经 `doc:operation` / `doc:cursor` 事件收发; `yjsUtils.js` 提供 `Yjs` 状态与 `base64` 的互转以及历史快照注水工具; `userColor.js` 根据 `userId` 哈希派生协作光标颜色。
- (4) **状态管理 (`store/authStore.js`)**: `Zustand` store, 包含 `user`、`accessToken`、`refreshToken` 三个字段及 `setAuth` (同步写 `localStorage`)、`logout` (清空 `state` 与 `localStorage`)、`loadFromStorage` (页面刷新后恢复登录态, 在 `App.jsx` 初始化时同步调用, 无竞态问题) 三个方法。
- (5) **路由 (`router/`)**: `react-router-dom v6` 配置五条路由, `ProtectedRoute` 检查 `accessToken`, 未登录时重定向至 `/login`。
- (6) **页面 (`pages/`)**: 各路由对应的顶层页面组件: `LoginPage` (登录)、`RegisterPage` (注册)、`HomePage` (协作项目列表)、`BoardPage` (白板编辑, 基于 `Fabric.js`)、`DocPage` (文档编辑, 集成 `TipTap` + `Yjs`)。

- (7) 组件 (**components/**): 可复用 UI 组件。公共组件: Navbar (顶部导航栏)、Loading (全屏加载)、Toast (全局消息提示)、ConfirmDialog (危险操作确认弹窗)。文档专属: DocEditor.jsx (TipTap 富文本编辑器)、DocCollabSidebar.jsx (评论/版本/成员侧边栏) 及编辑器浮层 (气泡菜单、斜杠命令菜单、表格操作菜单)。
- (8) 自定义 Hook (**hooks/**): useDocCollaboration.js 负责创建 Y.Doc、SocketIoYjsProvider、Y.UndoManager, 加入协作房间, 并从 Awareness 派生去重后的在线用户列表, 是文档协作功能的核心入口。

2. 模块之间的调用关系

后端调用链路:

HTTP 请求

```

└ Express App (app.js)
  └ 中间件链: requestLogger → CORS → auth (受保护路由)
    └ routes/ (声明路径与 HTTP 方法)
      └ controllers/ (提取 req 参数, 调用 service, 返回响应)
        └ services/ (业务逻辑, 调用 Prisma 查库, 抛 AppError)
          └ Prisma → PostgreSQL
  
```

WebSocket 连接

```

└ Socket.io 初始化 (socket/index.js)
  └ io.use(): JWT 连接鉴权, 挂 socket.data.userId
    └ handlers.js (为每条连接注册业务事件)
      └ 权限校验: 调用 docService.assertDocumentAccess
        └ / boardService.assertBoardReadable
      └ broadcastToRoom (排除发送者, 广播给房间其他成员)
  
```

前端调用链路:

App.jsx

```

└ authStore.loadFromStorage(): 同步恢复登录态
└ useSocket: 监听连接状态, 渲染全局横幅
└ router/index.jsx
  └ LoginPage / RegisterPage → api/auth.api.js (HTTP)
  └ ProtectedRoute (检查 accessToken)
    └ HomePage → api/board.api.js / api/doc.api.js (HTTP)
    └ BoardPage
      └ api/board.api.js (初始加载/保存)
      └ useSocket (joinRoom / emit board:sync / on board:sync)
      └ Fabric.js 画布渲染与事件监听
    └ DocPage
      └ api/doc.api.js (初始加载/保存/侧边栏)
      └ useDocCollaboration
        └ 创建 Y.Doc + SocketIoYjsProvider
          └ collab/socketIoYjsProvider.js → useSocket
  
```

| └ 派生在线用户列表
└ DocEditor (TipTap + Collaboration/CollaborationCursor 扩展)

(二) 核心业务流程设计

(1) 用户注册与登录流程

用户注册时,前端向 `POST /api/auth/register` 提交 `username`、`email`、`password`; 服务端依次进行格式校验(邮箱正则、用户名长度不超 32 字符、密码不少于 8 位)、唯一性校验(查库判断邮箱或用户名是否已注册),通过后用 `bcrypt (salt rounds = 10)` 对密码进行哈希存储,将用户记录写入 `User` 表,返回 201 与用户信息(不含密码哈希)。

用户登录时,前端向 `POST /api/auth/login` 提交邮箱与密码;服务端按邮箱查找用户,用 `bcrypt.compare` 验证密码哈希,检查账号是否被封禁,全部通过后签发 `accessToken` 与 `refreshToken` (有效期均通过环境变量配置)并返回。前端通过 `authStore.setAuth()` 将登录态持久化到 `localStorage`,后续每次 HTTP 请求由 `axios` 拦截器自动附带 `Authorization: Bearer <accessToken>`。

当 `accessToken` 过期时,`axios` 响应拦截器捕获错误码 40101,调用 `POST /api/auth/refresh` 用 `refreshToken` 换取新 `accessToken`;若 `refreshToken` 也已失效,则自动清理登录态并跳转至 `/login`。

(2) 文档实时协作流程

1. 用户打开文档页时,前端先通过 `GET /api/docs/:id` 拉取文档详情,获取 `content_data.yjs_state` (base64 编码的 Yjs 全量状态快照)。
2. `useDocCollaboration Hook` 创建本地 `Y.Doc` 实例与 `SocketIoYjsProvider`,并通过 `useSocket.joinRoom(itemId)` 触发 `Socket join` 事件加入协作房间(服务端校验 `viewer` 权限)。
3. 用 `applyPersistedYjsState(ydoc, yjs_state)` 对文档进行注水(`origin` 标记为 `'persisted'`,不触发 `Socket` 广播),完成文档初始化渲染。
4. 此后每次本地编辑触发 `Y.Doc` 的 `update` 事件, `Provider` 将增量 `update` 编码为 base64 后通过 `doc:operation` 事件发给服务端;服务端校验 `editor` 权限后,调用 `broadcastToRoom` 广播给房间内其他成员(排除发送者)。
5. 远端成员收到 `doc:operation` 事件后, `Provider` 调用 `Y.applyUpdate` 将增量无冲突合并到本地 `Y.Doc`, `TipTap` 编辑器自动更新视图。
6. 协作光标通过 `doc:cursor` 事件同步 `Awareness` (包含用户名、颜色、光标位置),所有在线成员的光标实时展示;在线用户列表由 `Awareness` 状态去重派生(同一用户多标签打开只计一次)。
7. 文档内容在两种时机持久化:停止编辑 2s 后防抖自动保存,或页面卸载前兜底保存,均调用 `PUT /api/docs/:id` 将 `Y.encodeStateAsUpdate(ydoc)` 的 base64 写入数据库。实时编辑全程不走 HTTP,仅通过 `Socket` 增量同步。

(3) 白板实时协作流程

1. 用户进入白板页,前端通过 `GET /api/boards/:id` 加载已保存的画布 JSON(`content_data.canvas`),交由 `Fabric.js` 初始化渲染。

2. 建立 Socket 连接后发送 join 事件加入房间，服务端查询 CollaborativeItem 类型后对白板执行 assertBoardReadable 权限校验。
3. 用户在画布上操作（绘制、移动、缩放、删除元素）时，Fabric.js 事件触发，前端将当前完整画布序列化为 JSON，通过 board:sync 事件广播给房间内其他成员，其他成员收到后调用 canvas.loadFromJSON 重新加载画布。
4. 用户光标位置通过 board:cursor 事件实时广播（节流处理），其他成员以带用户标识的小圆点展示光标。
5. 停止操作 1.2s 后触发防抖自动保存，调用 PUT /api/boards/:id 将画布 JSON 落库，editor 及以上权限方可执行保存操作。

（4）版本快照与回滚流程

1. 用户点击「创建版本」，前端调用 POST /api/docs/:id/versions，将当前 Yjs 全量状态（base64）、文档标题与用户自定义标签写入 Version 表；服务端保留最近 50 条版本记录，自动删除更早的快照，并通过 Socket 广播 doc:sidebar-changed 通知其他成员刷新版本列表。
2. 用户点击「回滚到此版本」，前端调用 POST /api/docs/:id/versions/:versionId/restore；服务端将该版本的 yjs_state 覆写到文档 content_data，然后主动向房间内所有成员（含操作者）广播 doc:version-restored 事件。
3. 全员收到 doc:version-restored 后，前端执行页面 reload，重新拉取文档详情并注水，从而完成所有在线成员同步到指定历史版本。

（三） 接口设计

系统提供两类接口：RESTful HTTP 接口（用于 CRUD 与数据持久化）和 Socket.io 事件接口（用于实时协作同步）。

统一响应格式：所有 HTTP 接口均返回如下 JSON 结构，前端 axios 拦截器统一解包与错误处理。

```

1 // 成功
2 { "code": 0, "data": <任意类型>, "message": "success" }
3 // 失败
4 { "code": <错误码>, "data": null, "message": "<错误描述>" }
```

表 1: HTTP 接口错误码约定

code	HTTP 状态码	含义
0	2xx	成功
40001	400	请求参数错误（字段缺失/格式不合法）
40101	401	未登录 / token 失效
40301	403	无权限操作
40401	404	资源不存在
40901	409	资源冲突（如邮箱已注册）
50000	500	服务器内部错误

认证接口 (`/api/auth`):

表 2: 认证模块 HTTP 接口

方法	路径	说明	是否鉴权
POST	<code>/api/auth/register</code>	注册, 请求体 {username, email, password}	否
POST	<code>/api/auth/login</code>	登录, 返回 accessToken + refreshToken + user	否
POST	<code>/api/auth/refresh</code>	用 refreshToken 换新 accessToken	否
GET	<code>/api/me</code>	获取当前用户信息	是
GET	<code>/api/health</code>	服务健康检查	否

白板接口 (`/api/boards`):

表 3: 白板模块 HTTP 接口

方法	路径	说明	最低权限
GET	<code>/api/boards</code>	获取当前用户可见白板列表	登录
POST	<code>/api/boards</code>	新建白板, 创建者自动成为 owner	登录
GET	<code>/api/boards/:id</code>	获取白板详情 (含画布 JSON)	viewer
PUT	<code>/api/boards/:id</code>	更新白板标题或画布内容	editor
DELETE	<code>/api/boards/:id</code>	删除白板 (级联删除关联数据)	owner
GET	<code>/api/boards/:id/versions</code>	获取版本快照列表	viewer
POST	<code>/api/boards/:id/versions</code>	创建版本快照	editor
POST	<code>/api/boards/:id/versions/:vid/restore</code>	回滚到指定版本	owner

文档接口 (`/api/docs`):

表 4: 文档模块 HTTP 接口

方法	路径	说明	最低权限
GET	/api/docs	获取当前用户可见文档列表	登录
POST	/api/docs	新建文档, 默认标题「未命名文档」	登录
GET	/api/docs/:id	获取文档详情 (含 yjs_state)	viewer
GET	/api/docs/:id/sidebar	聚合侧边栏数据 (评论/版本/成员/统计)	viewer
PUT	/api/docs/:id	保存文档标题或 Yjs 全量快照	editor
DELETE	/api/docs/:id	删除文档 (级联删除评论/版本/权限)	owner
GET	/api/docs/:id/comments	获取评论列表 (含回复树)	viewer
POST	/api/docs/:id/comments	发表锚点评论 (position.from/to 必填)	editor
POST	/api/docs/:id/comments/:cid/replies	回复某条评论	editor
PATCH	/api/docs/:id/comments/:cid/resolve	标记评论解决/未解决	editor
GET	/api/docs/:id/versions	获取版本快照列表	viewer
POST	/api/docs/:id/versions	创建版本快照 (含 yjs_state)	editor
POST	/api/docs/:id/versions/:vid/restore	回滚到指定版本并广播全员 reload	editor
GET	/api/docs/:id/members	获取成员列表 (含角色)	viewer
POST	/api/docs/:id/members/invite	按邮箱或用户名邀请成员	owner
PUT	/api/docs/:id/members/:uid	调整成员权限 (viewer/editor)	owner
DELETE	/api/docs/:id/members/:uid	移除成员	owner

Socket.io 事件接口:

所有 WebSocket 连接须在握手时于 `handshake.auth.token` 携带 `accessToken`; 服务端在 `io.use()` 中校验后将 `userId` 挂载到 `socket.data`。房间命名规范为 `item:<itemId>`, 一个协作项目对应一个 Socket 房间。

表 5: 基础 Socket 事件

事件名	方向	参数与说明	备注
join	C→S	{ itemId }, 加入协作房间, 服务端按 item 类型分发权限校验	—
leave	C→S	{ itemId }, 离开协作房间	—
ping	C→S	无参数, 连通性检测	—
pong	S→C	{ time }, 返回当前服务端时间戳	—
user:joined	S→C	{ userId, itemId }, 广播某用户加入房间	—
user:left	S→C	{ userId, itemId }, 广播某用户离开房间	—

表 6: 文档协作 Socket 事件

事件名	方向	说明	最低权限
doc:operation	C↔S	Yjs 增量 update (base64), 广播给房间其他成员	editor
doc:cursor	C↔S	Awareness 协作光标同步 (base64)	viewer
doc:title-changed	C↔S	标题变更, 广播给房间其他成员	editor
doc:sidebar-changed	C↔S	评论/版本变更, 通知其他成员静默刷新侧边栏	viewer
doc:version-restored	S→C	版本回滚成功, 全员 (含操作者) 需 reload 文档	—
doc:error	S→C	权限校验失败, 回传 { itemId, code, message }	—

表 7: 白板协作 Socket 事件

事件名	方向	说明
board:sync	C↔S	画布整体 JSON 广播, 其他成员收到后重载画布
board:draw	C↔S	单次笔迹操作广播 (轻量增量, 降低带宽占用)
board:cursor	C↔S	多人光标位置实时同步, 参数为 { itemId, x, y }
board:error	S→C	白板权限校验失败, 回传错误码与描述

三、 UML 设计

(一) 用例图 (Use Case Diagram)

系统用例图从外部参与者的视角描述了 CollabPlatform 的功能边界与权限范围, 如图 1 所示。

Actor 继承链体现最小权限原则, 上层 Actor 自动继承下层所有权限:

访客 $\subset$ 登录用户 $\subset$ 查看者 $\subset$ 编辑者 $\subset$ 所有者

系统共划分为四个功能包, Actor 连接到功能包 (而非单个用例), 以反映“角色可访问的功能域”这一粒度。

表 8: 用例功能包说明

功能包	主要职责	最低 Actor
认证模块	注册账号、用户登录、Token 自动刷新	访客
项目管理	创建 / 重命名 / 删除项目, 邀请成员与权限调整	登录用户
文档协作	查看、编辑、评论、版本快照与恢复、导出	查看者
白板协作	查看、绘图、元素操作、光标同步、版本快照与导出	查看者

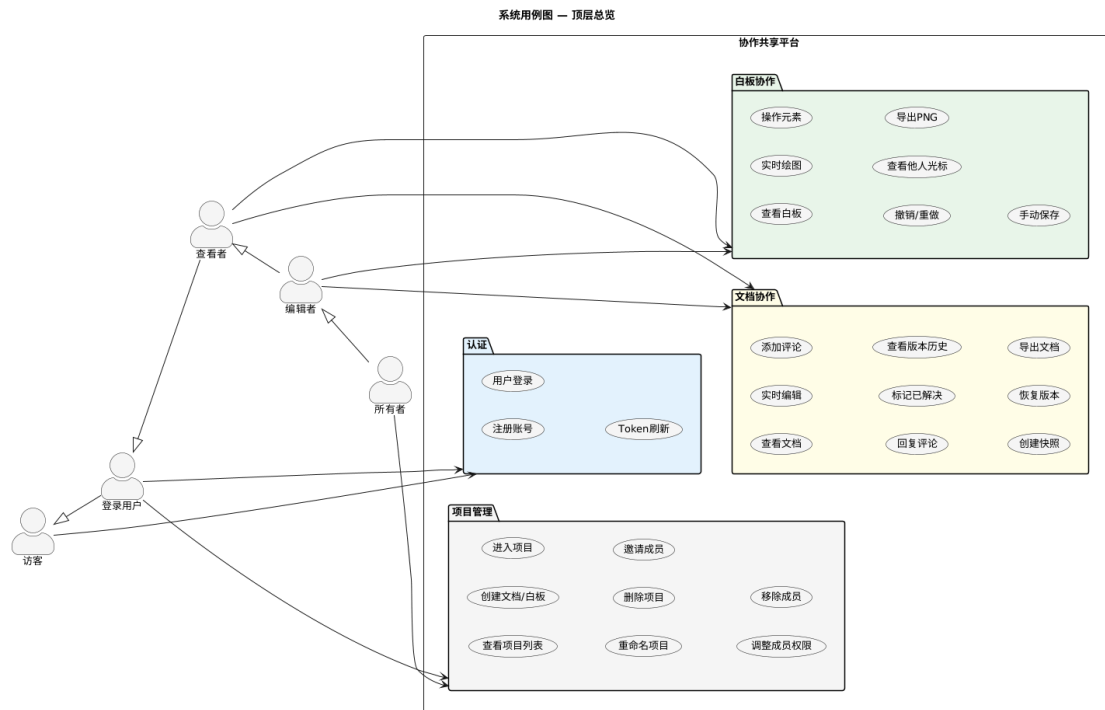


图 1: 系统用例图—顶层总览

(二) 类图 (Class Diagram)

1. 领域模型类图

领域模型类图 (图 2) 描述了数据库层的核心实体及其关系, 与 Prisma schema 直接对应。

核心设计: CollaborativeItem 通过 type 枚举字段 (Document/Whiteboard) 统一管理两类协作项目, 权限 (Permission)、版本 (Version) 与评论 (Comment) 作为通用扩展能力被两类项目共用, 避免重复建模。Permission 为 User 与 CollaborativeItem 的 N:M 关联实体, 携带 role 字段表达三级权限; Comment 通过 parent_id 自关联实现两级回复树结构。

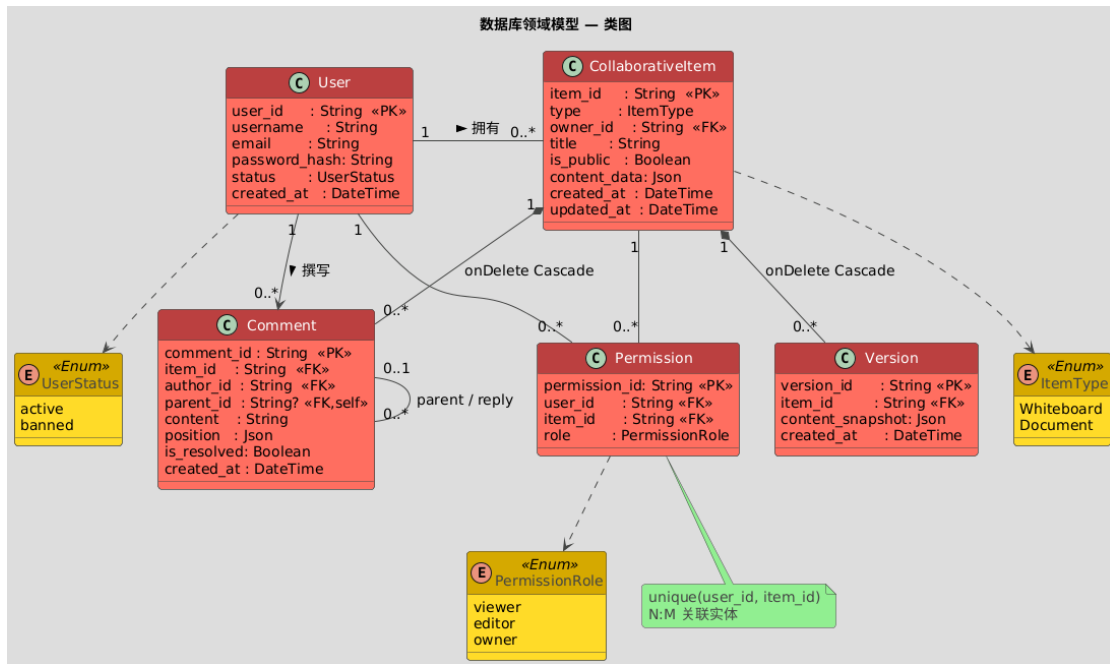


图 2: 领域模型类图（对应 Prisma Schema）

2. 服务层类图

服务层类图（图 3）描述了后端四个 Service 模块的公开接口及其对 Prisma ORM 的共同依赖。图中布局为：AuthService / DocService 位于左侧，PrismaClient 居中，ItemService / BoardService 位于右侧。

可见性约定：+ 表示由 Controller 或 Socket Handler 调用的公开方法；~ 表示仅在 Service 内部调用的鉴权断言方法（如 assertDocumentAccess、assertBoardWritable），不对外暴露 HTTP 接口。



图 3: 服务层类图 (Service 层接口与依赖关系)

(三) 顺序图 (Sequence Diagram)

1. 文档实时协作编辑 (UC-05)

图 4 描述了文档实时协作的完整数据流，涵盖 HTTP 初始化、Yjs 注水、Socket 加入协作房间，以及多用户 CRDT 增量同步的全过程。

系统设置了三道防循环机制以保证 CRDT 语义正确性：

- (1) origin='persisted': 注水时跳过广播，不触发 doc:operation，防止历史内容被误广播。
- (2) except senderSocketId: 服务端广播排除发送者，防止本地编辑自回显。
- (3) origin=provider: 接收方 applyUpdate 时检测 origin，避免二次广播产生回环。

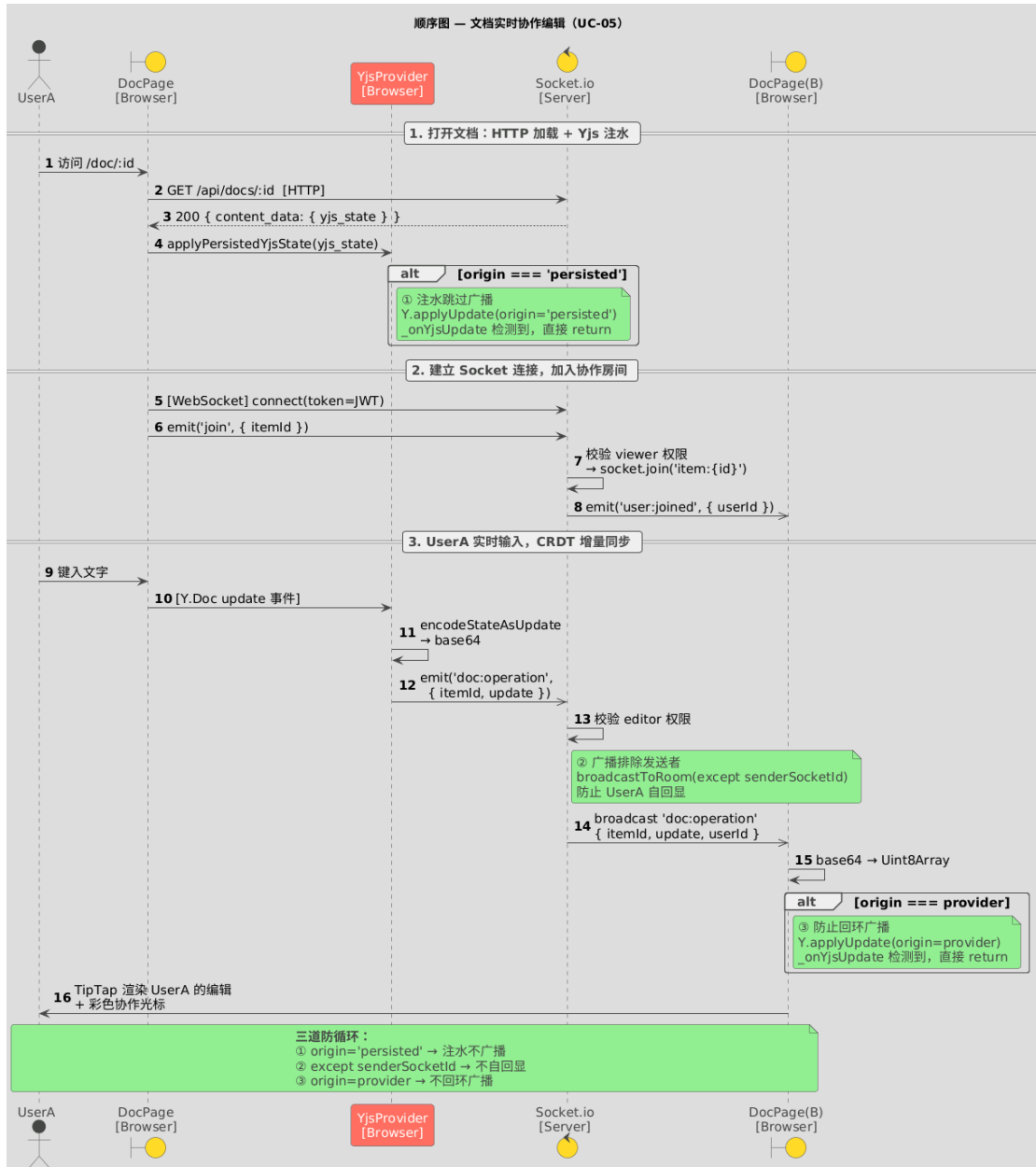


图 4: 顺序图—文档实时协作编辑 (UC-05)

2. 版本历史与回滚 (UC-07)

图 5 描述了 Editor 触发版本恢复时 HTTP 与 WebSocket 的协同流程。

关键设计决策: HTTP 负责持久化 (写数据库, 有事务保障, 结果明确); WebSocket 负责广播通知 (将 doc:version-restored 事件发给房间内所有成员, 含操作者本身, 不设 exceptSocketId)。全员收到事件后均执行页面 reload, 重新拉取文档并注水, 从而保证所有在线成员状态完全一致, 彻底消除状态分叉风险。

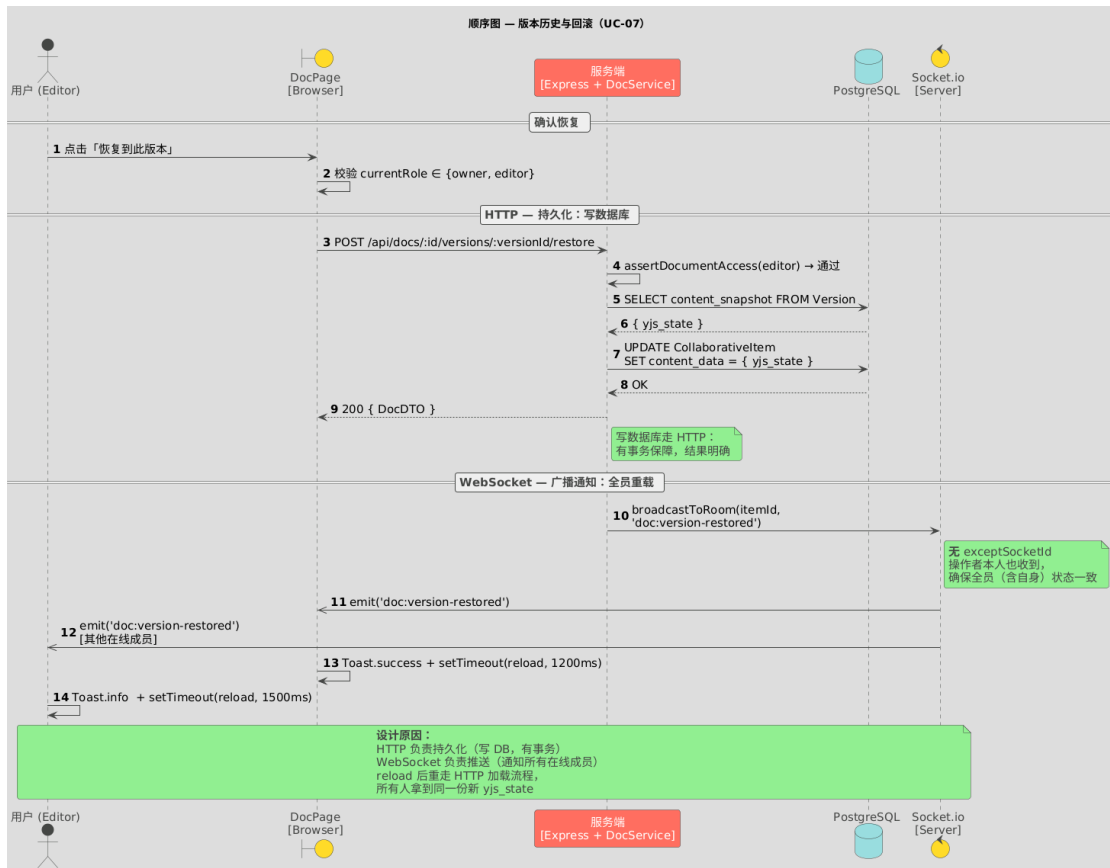


图 5: 顺序图—版本历史与回滚（UC-07）

3. 白板实时协作（BoardPage）

图 6 描述了白板协作的加载、实时绘图同步、光标同步与版本快照保存的完整流程。与文档协作的主要差异对比如表 9 所示。

表 9: 文档协作与白板协作的技术方案对比

维度	文档（Yjs / TipTap）	白板（Fabric.js）
同步数据	Yjs 增量 CRDT update (base64)	Fabric.js canvas 全量 JSON 快照
广播事件	doc:operation	board:sync (120ms 防抖)
光标同步	doc:cursor (Awareness)	board:cursor (50ms 节流，绝对坐标)
持久化触发	防抖 2s → PUT	防抖 1200ms → PUT
自动版本	无	每 5s 检查，有变更且距上次 ≥ 30s 才创建
回环防护	applyingRemote origin 检测	applyingRemote 标志位

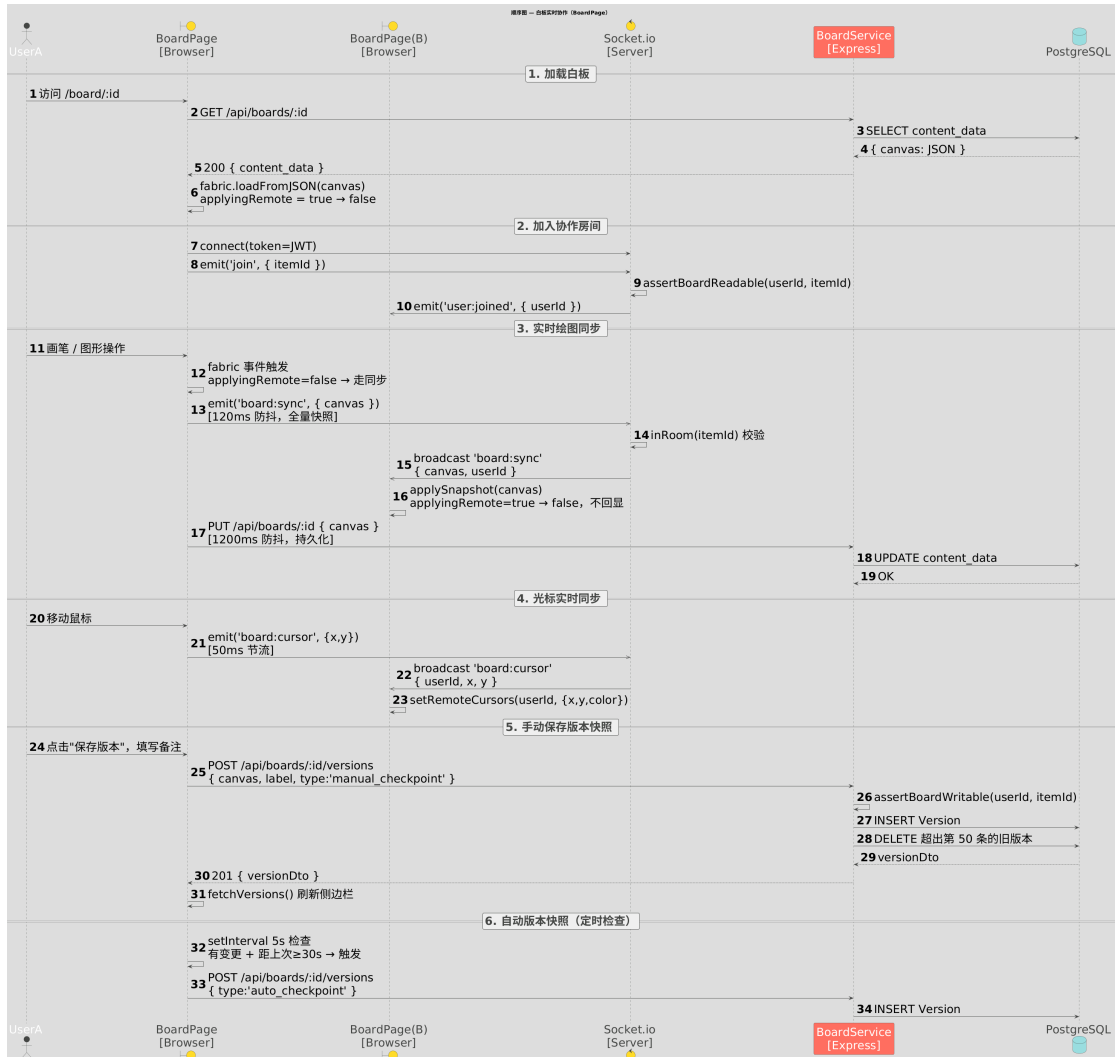


图 6: 顺序图—白板实时协作 (BoardPage)

(四) 状态图与部署图

1. 文档协作会话状态图

图 7 描述了 DocPage.jsx 的完整生命周期状态机，覆盖从初始加载到页面卸载的全过程。

启动三阶段：加载元数据（HTTP GET）→ 建立 Socket 连接（JWT 鉴权）→ 注水 Yjs 快照 → 就绪。

就绪状态为复合状态，内部包含四个子状态：空闲（Idle）、编辑中（Editing，防抖计时器运行）、防抖保存中（Saving，PUT 落库）、快照创建中（Snapshotting）。

两个关键设计：

- (1) **离线编辑：**Socket 断开后进入 Offline 状态，Yjs CRDT 在本地内存继续工作，内容不丢失，重连后自动无冲突合并——这是选用 Yjs 而非 OT 算法的核心价值。
- (2) **兜底保存（Unloading）：**页面关闭或路由切换时，useEffect cleanup 触发 fire-and-forget PUT，是防止数据丢失的最后一道防线，即便 2s 防抖计时器尚未到期也能保存最新状态。

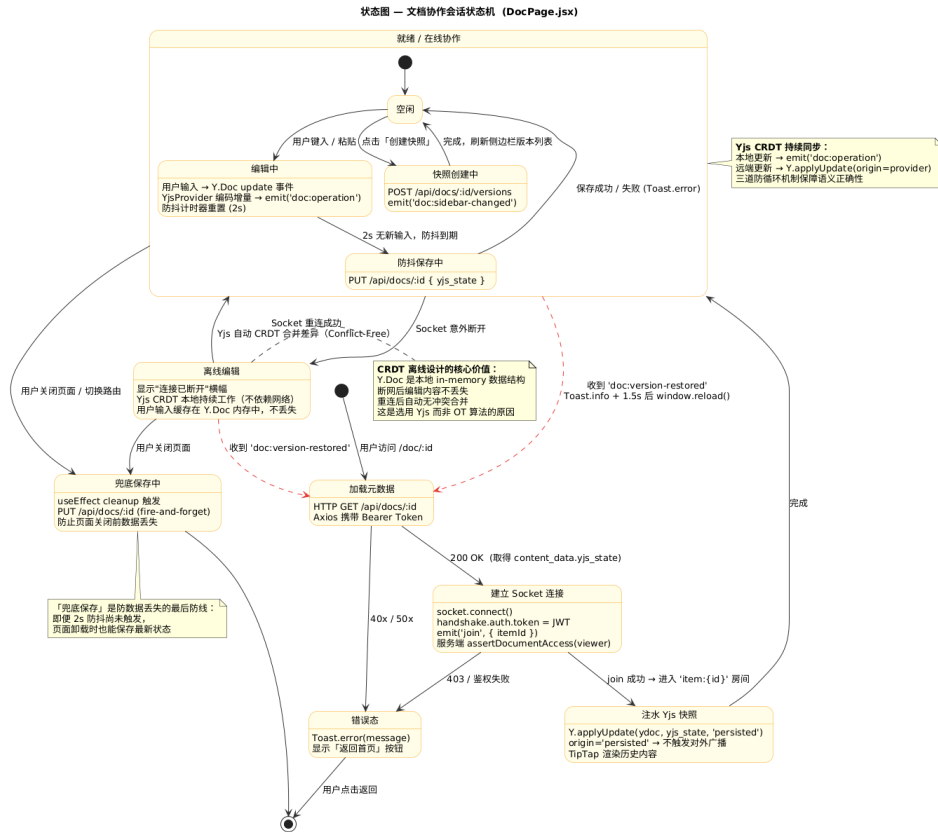


图 7: 状态图—文档协作会话状态机 (DocPage.jsx)

2. 部署图

图 8 描述了系统的物理部署拓扑，采用三层架构：用户浏览器、Node.js 应用服务器（端口 3000）与数据层（PostgreSQL + 可选 Redis）。

Socket.io Server 与 Express 共享同一 HTTP 实例，无需额外端口。PostgreSQL 连接通过 PgBouncer 连接池（端口 6543），连接串须加 ?pgbouncer=true 并禁用 Prepared Statement。Redis 未配置时自动降级为 noop Proxy，不影响核心功能。

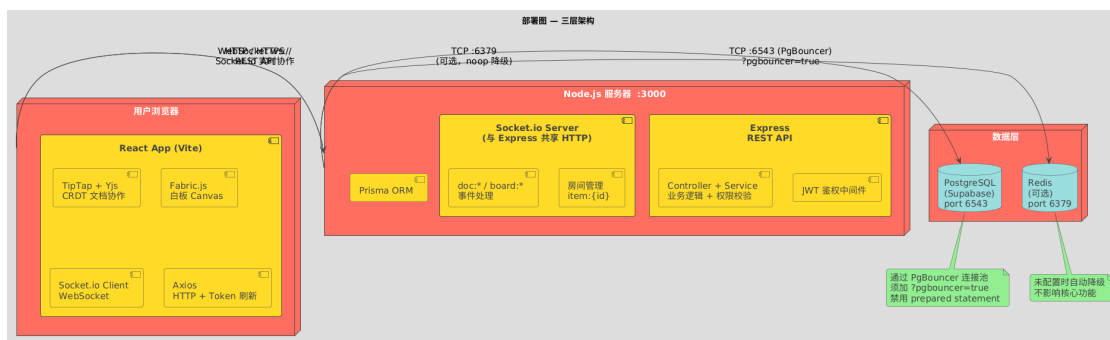


图 8: 部署图—三层架构 (Browser / Node.js / 数据层)

四、 数据库设计

(一) 数据库 E-R 图

本项目采用 PostgreSQL 关系型数据库，通过 Prisma ORM 进行模型管理，共包含 5 张核心数据表。E-R 图如图 9 所示，图中采用 Crow’s Foot 记法表示关系基数（|| 表示恰好一个，o{ 表示零个或多个，|o 表示零个或一个）。

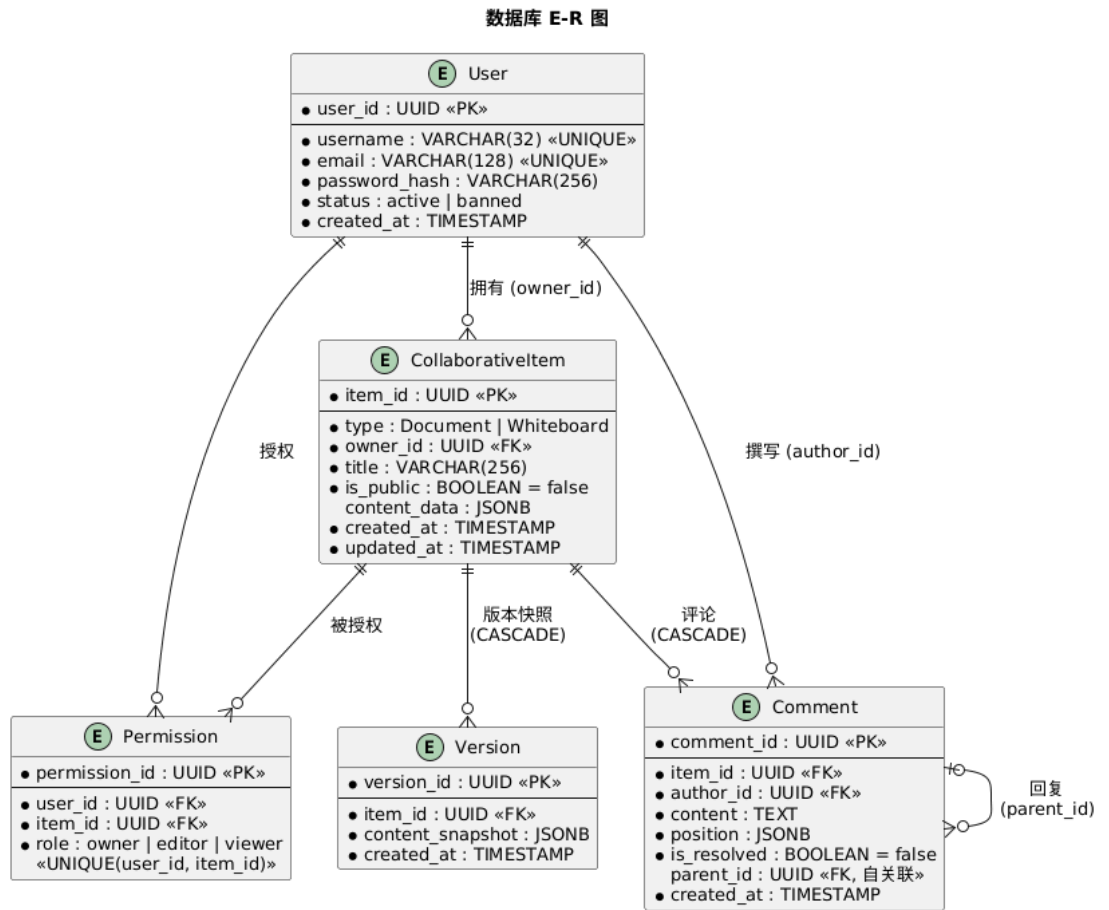


图 9: 数据库 E-R 图（Crow’s Foot 记法）

各实体间关系说明如表 10 所示。

表 10: 实体关系概述

关系	类型	说明
User → CollaborativeItem	1 : N	一个用户可拥有多个协作项目 (owner_id)
User × CollaborativeItem → Permission	N : M	权限表作为关联实体, 记录用户对项目的角色
CollaborativeItem → Version	1 : N	一个项目可有多条版本快照, 级联删除
CollaborativeItem → Comment	1 : N	一个项目可有多条评论, 级联删除
User → Comment	1 : N	一个用户可撰写多条评论 (author_id)
Comment → Comment	0..1 : N	评论自关联, parent_id 指向父评论, 实现回复树

核心设计决策: CollaborativeItem 用 type 枚举字段统一管理文档与白板, 权限 (Permission)、版本 (Version) 与评论 (Comment) 作为通用扩展能力被两类项目共用, 避免重复建模。

(二) 主要数据表设计

1. 用户表 User

存储平台注册用户的基础信息与账号状态。

表 11: User 表字段设计

字段名	数据类型	约束	说明
user_id	VARCHAR(36)	主键, UUID 自动生成	用户唯一标识
username	VARCHAR(32)	NOT NULL, UNIQUE	用户名, 全局唯一
email	VARCHAR(128)	NOT NULL, UNIQUE	邮箱, 用于登录
password_hash	VARCHAR(256)	NOT NULL	bcrypt 哈希 (cost=10), 明文不入库
status	ENUM	NOT NULL, DEFAULT active	active / banned
created_at	TIMESTAMP	NOT NULL, DEFAULT now()	注册时间

2. 协作项目表 CollaborativeItem

统一存储文档与白板两类协作项目, 是整个系统的核心实体。

表 12: CollaborativeItem 表字段设计

字段名	数据类型	约束	说明
item_id	VARCHAR(36)	主 键, UUID 自动生成	项目唯一标识
type	ENUM	NOT NULL	Document / Whiteboard
owner_id	VARCHAR(36)	NOT NULL, 外键 → User.user_id	项目所有者
title	VARCHAR(256)	NOT NULL	项目标题
is_public	BOOLEAN	NOT NULL, DEFAULT false	是否公开（预留字段）
content_data	JSONB	可空	文档存 {yjs_state}, 白板存 {canvas}
created_at	TIMESTAMP	NOT NULL, DEFAULT now()	创建时间
updated_at	TIMESTAMP	NOT NULL, 自动更新	最后修改时间, Prisma @updatedAt 维护

3. 权限表 **Permission**

实现用户与项目之间的 N:M 多对多关系，同时记录角色级别，是权限控制的核心。

表 13: Permission 表字段设计

字段名	数据类型	约束	说明
permission_id	VARCHAR(36)	主键, UUID 自动生成	权限记录标识
user_id	VARCHAR(36)	NOT NULL, 外键 → User.user_id	被授权用户
item_id	VARCHAR(36)	NOT NULL, 外键 → CollaborativeItem.item_id, CASCADE	目标项目
role	ENUM	NOT NULL	owner / editor / viewer
唯一约束: UNIQUE (user_id, item_id), 同一用户对同一项目只有一条权限记录			

权限继承关系为 owner > editor > viewer, 服务层通过数值映射 {viewer:1, editor:2, owner:3} 进行比较校验, 不依赖数据库触发器。

4. 版本快照表 Version

存储项目的历史版本内容, 文档与白板均使用, 支持版本回滚。

表 14: Version 表字段设计

字段名	数据类型	约束	说明
version_id	VARCHAR(36)	主键, UUID 自动生成	版本唯一标识
item_id	VARCHAR(36)	NOT NULL, 外键 → CollaborativeItem.item_id, CASCADE	所属项目
content_snapshot	JSONB	NOT NULL	版本内容快照
created_at	TIMESTAMP	NOT NULL, DEFAULT now()	快照创建时间

content_snapshot 结构约定: 文档版本为 {yjs_state, title, type, created_by}; 白板版本为 {canvas, title, type, created_by, label}, 其中 type 为 "auto_checkpoint" (定时自动, 每 30s 检查一次) 或 "manual_checkpoint" (用户手动保存)。每个项目最多保留最近 50 条版本, 超出后由服务层在写入新版本时自动删除最旧记录。

5. 评论表 Comment

存储文档的锚点评论及其回复, 仅文档类型项目使用。

表 15: Comment 表字段设计

字段名	数据类型	约束	说明
comment_id	VARCHAR(36)	主 键, UUID 自动生成	评论唯一标识
item_id	VARCHAR(36)	NOT NULL, 外键 → CollaborativeItem.item_id, CASCADE	所属文档
author_id	VARCHAR(36)	NOT NULL, 外键 → User.user_id	评论作者
content	TEXT	NOT NULL	评论正文, 无长度上限
position	JSONB	NOT NULL	锚点位置: {from, to, selected_text}
is_resolved	BOOLEAN	NOT NULL, DEFAULT false	是否已标记解决
parent_id	VARCHAR(36)	可空, 外键 (自关联) → Comment.comment_id, CASCADE	父评论 ID, null 为顶层评论
created_at	TIMESTAMP	NOT NULL, DEFAULT now()	创建时间

parent_id 为 NULL 表示顶层评论, 非空则为回复, 指向所回复的顶层评论 ID (两级结构, 不支持无限嵌套)。position 字段记录用户在文档中选中文本的 TipTap 节点偏移量, 用于侧边栏将评论与原文锚点对应展示。

6. 枚举类型说明

表 16: 数据库枚举类型

枚举名	可选值	用途
UserStatus	active、banned	账号状态，banned 状态拒绝登录
ItemType	Document、Whiteboard	区分项目类型，决定 content_data 结构
PermissionRole	owner、editor、viewer	三级权限角色，用于 Socket 与 REST API 双重校验

五、 其他内容

（一） 安全性设计

- (1) **认证与会话**：系统使用 JWT (accessToken + refreshToken) 进行无状态认证；前端在请求拦截器中自动附带 Authorization: Bearer <token>，后端在 authMiddleware 中校验并将用户标识挂载到 req.user。
- (2) **密码安全**：用户密码在服务端使用 bcrypt 进行哈希存储，不以明文形式落库。
- (3) **权限控制**：采用 Owner/Editor/Viewer 三级权限模型。HTTP API 在 service 层做读写权限校验；Socket 层在 join 与协作事件处理中进行权限校验，避免绕过 HTTP 直接通过 WebSocket 读写数据。
- (4) **跨域与最小暴露**：后端 CORS 仅放行配置的前端域名(CLIENT_ORIGIN)，并设置 credentials；统一错误码返回，避免泄露内部堆栈信息（生产环境不输出详细错误）。

在生产部署时，建议进一步补充：HTTPS/WSS、访问频率限制（可基于 Redis 计数）、更严格的输入校验与审计日志等。

（二） 性能优化设计

- (1) **网络层**：实时协作使用房间广播，避免全局广播；对光标等高频事件采用节流；对持久化保存采用防抖，减少频繁落库。
- (2) **协作层**：基于 Yjs 的增量 update 同步，仅传输差异数据，降低带宽占用并提升多人并发下的合并效率。
- (3) **数据层**：Prisma 查询使用字段选择 (select) 与必要的索引/约束，降低无效字段读取；版本快照采用“只保留最近 N 条”的策略避免表无限增长。
- (4) **前端渲染**：将连接状态、加载状态、侧边栏等 UI 状态与业务状态分离，减少不必要的重渲染；大型模块可按路由拆分实现按需加载（后续优化方向）。

（三） 可扩展性设计

- (1) **模块化目录与职责边界**：前端按 api/components/pages/hooks/collab 分层，后端按 routes/controllers/services 分层；新增功能通常只需要在对应层新增文件并在路由入口注册。

- (2) **统一协作模型**: CollaborativeItem 作为统一资源抽象, 权限 (Permission)、版本 (Version)、评论 (Comment) 为通用扩展点, 新协作类型可复用这些能力。
- (3) **统一协议与常量**: 错误码与 Socket 事件名集中定义, 减少多人协作时的命名分歧与联调成本。

(四) 异常处理机制

- (1) **后端**: 业务层通过抛出 `AppError(httpStatus, code, message)` 表达可预期错误; 全局错误中间件统一捕获并按标准格式返回。对未知错误返回 50000, 开发环境可输出日志用于定位。
- (2) **前端**: axios 响应拦截器统一处理业务错误码; token 失效 (40101) 时自动清理登录态并跳转登录页; 其余错误通过 Toast 统一提示。
- (3) **实时通道**: Socket 连接层使用 token 鉴权; 在权限不足或参数错误时通过 `doc:error/board:error` 等事件回传, 避免静默失败。

(五) 日志与监控方案

- (1) **请求日志**: 后端使用 morgan 记录请求方法、路径、状态码与耗时, 为性能分析与问题回溯提供依据。
- (2) **错误日志**: 全局错误处理中间件在开发环境输出堆栈, 便于快速定位; 生产环境应接入结构化日志 (如 Winston/Pino) 并输出到文件或日志平台。
- (3) **运行监控 (建议)**: 可进一步加入进程级健康检查、数据库连接监控、Socket 在线连接数统计, 以及 Prometheus + Grafana 等指标体系, 便于上线后观察稳定性与容量。